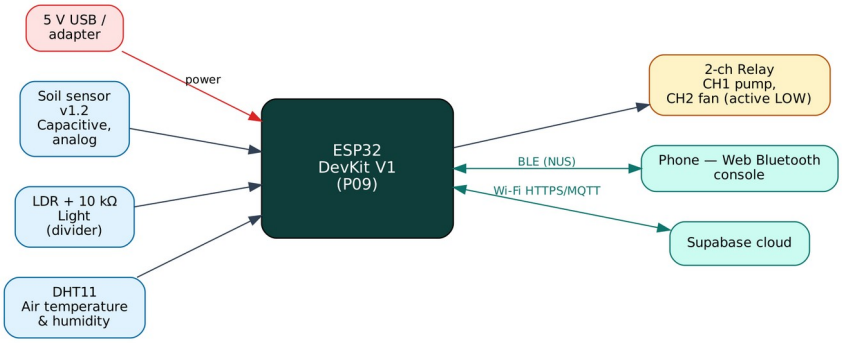
A PROJECT REPORT ON

Smart Greenhouse with Cloud-Configurable Irrigation

IoT project using ESP32 with Bluetooth Low Energy and Wi-Fi (Supabase)



Project: P09 · Advanced level

Peripherals: Capacitive soil moisture sensor v1.2, DHT11, LDR, 2-channel relay (pump + fan)

Submitted by: _____

Under the guidance of: _____

Department / Institution: _____

October 2026

Certificate

This is to certify that the project report entitled **“Smart Greenhouse with Cloud-Configurable Irrigation”** is a bonafide record of the work carried out by _____ (Roll No. _____) in partial fulfilment of the requirements for the award of _____ in _____ during the academic year _____.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

Acknowledgement

I express my sincere gratitude to my project guide _____ for continuous guidance and support, to the Head of the Department _____, and to all faculty members and friends who helped me complete this project successfully.

Contents

Abstract 6

1. Introduction 7

1.1 Problem Statement 7
1.2 Objectives 7
1.3 Scope of the Project 7
1.4 Organisation of the Report 7

2. Existing System and Proposed System 8

2.1 Existing Systems 8
2.2 Proposed System 8
2.3 Why Both BLE and Wi-Fi? 8

3. System Design 9

3.1 Block Diagram 9
3.2 System Architecture 9
3.3 Hardware Components and Cost 9
3.4 ESP32 DevKit V1 10
3.5 Working Principle of Components 10

4. Circuit Design 11

4.1 Circuit Diagram 11
4.2 Pin Connections 11
4.3 Interfacing and Power Notes 12

5. Data Flow Diagrams 13

5.1 DFD Level 0 (Context Diagram) 13
5.2 DFD Level 1 14
5.3 Data Dictionary 15

6. Software Design and Implementation 16

6.1 Development Environment 16
6.2 Firmware Architecture 16
6.3 Working of the System 16
6.4 Flowchart 17
6.5 BLE Communication Protocol 17
6.6 Cloud Integration (Supabase) 18
6.7 Configurable Settings 18

7. Setup and Deployment Guide 19

7.1 Hardware Assembly 19
7.2 Flashing the Firmware 19
7.3 Cloud Setup (Supabase) 19
7.4 BLE App Setup (Web Bluetooth Console) 20
7.5 First-Run Checklist 21

8. Testing and Results 22

8.1 Expected Output 22
8.2 Troubleshooting 22

9. Discussion 23

9.1 Advantages 23
9.2 Limitations 23
9.3 Safety Precautions 23
9.4 Security Considerations 23
9.5 Future Scope 23

10. Conclusion **24**

References **25**

Appendix A: Project Source Code **26**

Appendix B: Shared Library Code **30**

Abstract

Irrigation by fixed timers wastes water and stresses plants, and greenhouses overheat quickly on sunny days. This project monitors soil moisture with a corrosion-resistant capacitive sensor, air temperature and humidity with a DHT11 and light with an LDR, and controls a water pump and a ventilation fan through a two-channel relay. Irrigation uses a burst-and-soak strategy: the pump runs in short bursts and waits for water to spread before measuring again, which prevents over-watering. The fan runs when temperature or humidity is too high. All readings and actions are logged to Supabase, and the thresholds are read from a settings row in the database, so they can be changed from anywhere in the Supabase table editor. BLE gives local monitoring, manual control and two-point soil calibration.

Keywords: ESP32, Internet of Things, Bluetooth Low Energy, Wi-Fi, Supabase, Capacitive soil moisture sensor v1.2, DHT11, LDR, 2-channel relay (pump + fan).

1. Introduction

The Internet of Things (IoT) connects physical devices — sensors and actuators — to networks so that data can be collected, analysed and acted on automatically. The ESP32 from Espressif Systems has made IoT affordable: a single ₹350 board combines a dual-core 240 MHz processor, Wi-Fi and Bluetooth Low Energy. This project uses both radios at once: BLE for direct, local interaction with a phone and Wi-Fi for cloud connectivity through Supabase, a free IoT/cloud platform.

1.1 Problem Statement

Small farmers and home gardeners either over-water (wasting water, causing root rot) or under-water (yield loss). Greenhouses need ventilation at the right time. Commercial controllers are expensive and need the operator on site to change settings.

1.2 Objectives

1. Measure soil moisture, temperature, humidity and light.
2. Irrigate automatically using burst-and-soak control between two moisture thresholds.
3. Ventilate automatically on high temperature or humidity with hysteresis.
4. Log readings and events to Supabase and read thresholds from the cloud.
5. Allow manual override and calibration from a phone over BLE.

1.3 Scope of the Project

One greenhouse zone or a group of pots/beds with similar water needs, a 5-12 V pump or solenoid valve and a fan. Multi-zone control and fertigation are outside the scope.

1.4 Organisation of the Report

Chapter 2 reviews existing approaches and describes the proposed system. Chapter 3 covers system design and components, Chapter 4 the circuit, Chapter 5 the data flow diagrams, Chapter 6 the software, Chapter 7 the step-by-step setup of hardware, cloud and app, Chapter 8 testing, Chapter 9 discussion and Chapter 10 the conclusion. Complete source code is given in the appendices.

2. Existing System and Proposed System

2.1 Existing Systems

Most soil-moisture projects use cheap resistive probes that corrode in weeks and switch the pump on until the sensor reads wet — but water takes time to spread, so the soil ends up over-watered. Thresholds are hard-coded.

2.2 Proposed System

The proposed system improves on these in the following ways:

- Capacitive sensor (no exposed metal) with two-point calibration stored in flash.
- Burst-and-soak state machine (15 s water, 120 s soak) to avoid overshoot.
- Thresholds stored in a Supabase table and fetched every minute; last values kept offline.
- Fan control by temperature OR humidity with hysteresis.

2.3 Why Both BLE and Wi-Fi?

Wi-Fi gives the device internet access for remote monitoring, history and control from anywhere, but it depends on a router and an internet connection, and the device needs the Wi-Fi password before it can connect. BLE needs no infrastructure: any modern phone can connect directly within about 10-30 m. Using both gives:

- **Provisioning** — Wi-Fi credentials and cloud keys are sent from the phone over BLE and stored in flash; nothing secret is hard-coded or uploaded to GitHub.
- **Resilience** — local monitoring and control keep working when the internet or router is down.
- **Remote access** — the cloud dashboard works from anywhere when the internet is available.
- **Low cost** — no extra hardware; both radios are built into the ESP32.

The ESP32 shares one 2.4 GHz radio between Wi-Fi and Bluetooth by time-division coexistence. For this to work the Wi-Fi modem-sleep mode must stay enabled, which the firmware ensures. NimBLE, a lightweight BLE stack, is used instead of the default Bluedroid stack to leave more RAM for TLS (HTTPS/MQTTS) connections.

3. System Design

3.1 Block Diagram

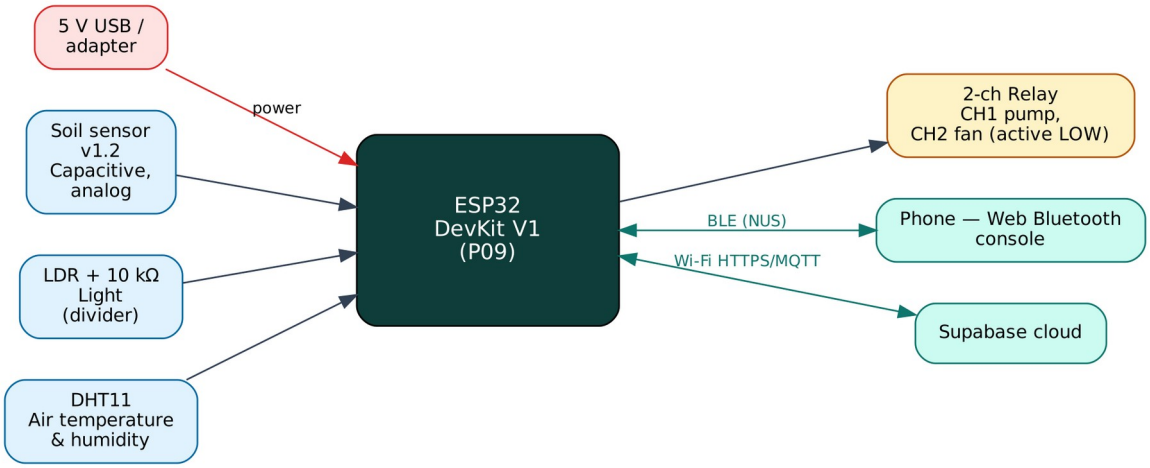


Figure 1: Block diagram of the Smart Greenhouse

The ESP32 is the central controller. Input devices (blue) feed measurements to it, output devices (amber) are driven by it, and two communication links (green) connect it to the user's phone over BLE and to the Supabase cloud over Wi-Fi. The board is powered from 5 V through its USB connector or VIN pin.

3.2 System Architecture

The system follows the standard three-layer IoT architecture:

- **Perception layer** — Capacitive soil moisture sensor v1.2, DHT11, LDR, 2-channel relay (pump + fan) connected to the ESP32 GPIO, ADC, I2C, SPI or UART.
- **Network layer** — BLE 4.2 GATT server (Nordic UART Service) and Wi-Fi 802.11 b/g/n with TLS-encrypted HTTPS REST to Supabase.
- **Application layer** — the Web Bluetooth console on the phone and the Supabase dashboard / database.

3.3 Hardware Components and Cost

#	Component	Qty	Specification	Cost
1	ESP32 DevKit V1 (ESP-WROOM-32, 30-pin)	1	Dual-core 240 MHz, Wi-Fi + BLE, 4 MB flash	₹350
2	Breadboard 830-point + jumper wires (M-M, M-F)	1	Prototyping and connections	₹150
3	Micro-USB data cable	1	Power + programming	₹60
4	Capacitive soil moisture sensor v1.2	1	Analog, 3.3-5 V, corrosion resistant	₹70
5	DHT11 temperature & humidity module	1	0-50 °C, 20-90 % RH	₹60
6	LDR 5 mm + 10 kΩ resistor	1	Light level	₹10
7	2-channel 5 V relay module	1	Pump + fan	₹80
8	5 V mini submersible pump + tube	1	Irrigation (demo)	₹90

#	Component	Qty	Specification	Cost
9	5 V / 12 V DC fan	1	Ventilation (demo)	₹60
	Total (approx.)			**₹930**

Table 1: Bill of materials with approximate Indian prices (Robu.in / Amazon.in / local market, 2026)

> Prices are indicative and vary by seller. The breadboard, cable and ESP32 can be reused across all ten projects, so the incremental cost of this project is about ₹370.

3.4 ESP32 DevKit V1

Feature	Value
Module	ESP-WROOM-32 (ESP32-D0WD), dual-core Xtensa LX6 up to 240 MHz
Memory	520 KB SRAM, 4 MB SPI flash
Wireless	Wi-Fi 802.11 b/g/n 2.4 GHz; Bluetooth 4.2 BR/EDR + BLE
I/O	25 usable GPIO on the 30-pin board, 12-bit ADC (ADC1: GPIO32-39), 2 × DAC, PWM on any output
Interfaces	3 × UART, 2 × I2C, 3 × SPI, I2S, touch, RMT
Supply	5 V via USB/VIN, on-board 3.3 V regulator (AMS1117)
Logic level	3.3 V — inputs are NOT 5 V tolerant

Table 2: Key specifications of the ESP32 DevKit V1

3.5 Working Principle of Components

Capacitive soil sensor

The probe forms a capacitor whose dielectric is the surrounding soil. Water has a high dielectric constant (~80) compared with dry soil (~4), so wetter soil increases capacitance. A 555 timer oscillator and RC filter turn this into a DC voltage that falls as moisture rises. No metal touches the soil, so it does not corrode.

Two-point calibration

Moisture % = (V_dry − V) / (V_dry − V_wet) × 100, where V_dry is measured with the probe in air and V_wet with it in water (caldry / calwet commands).

DHT11 & LDR

The DHT11 measures air temperature and humidity; the LDR divider measures light for logging and future shading control.

Supabase settings row

A single-row table greenhouse_settings (id = 1) holds soil_min, soil_max, temp_max, hum_max and auto_mode. Editing it in the Supabase dashboard changes the controller's behaviour within a minute.

4. Circuit Design

4.1 Circuit Diagram

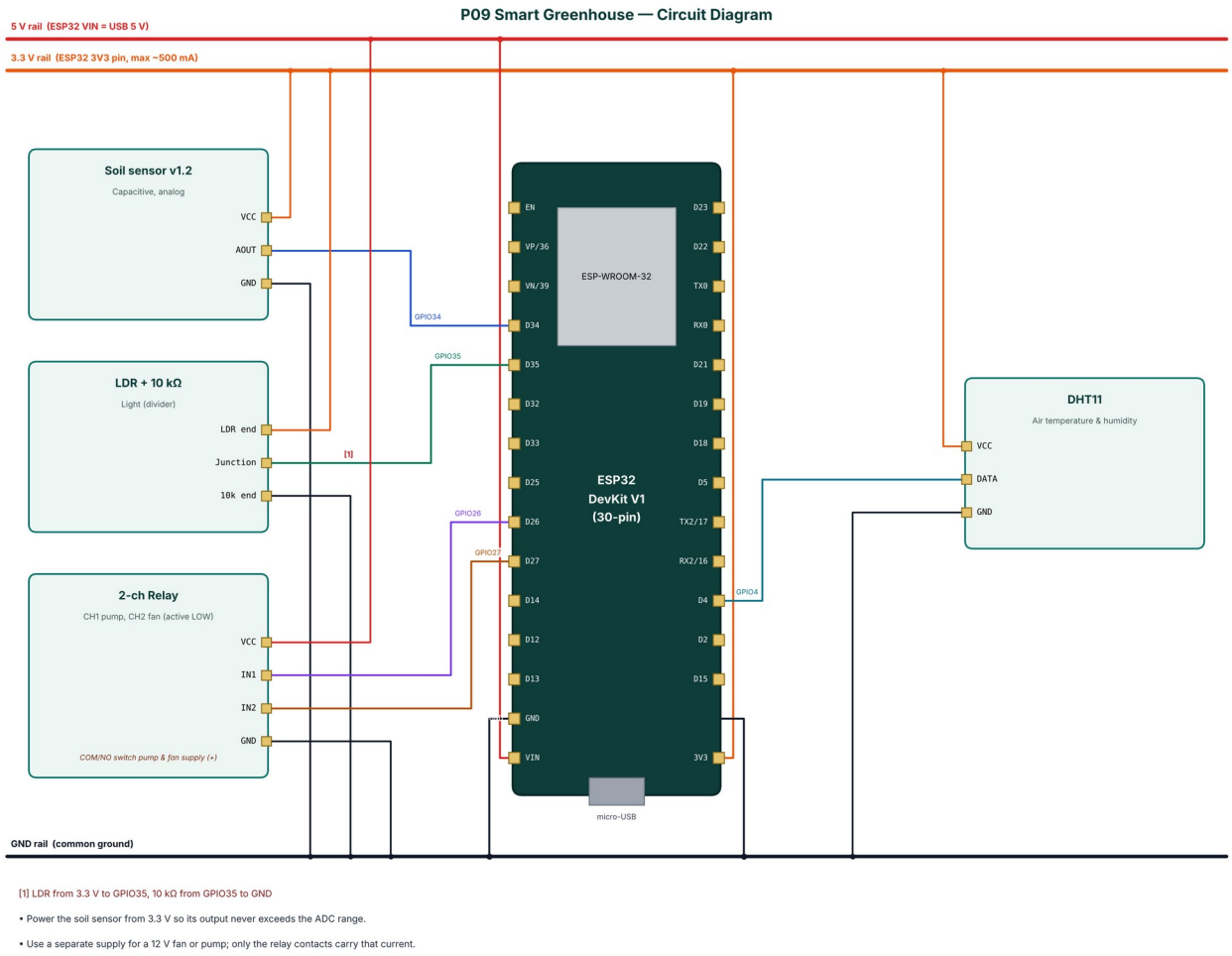


Figure 2: Circuit diagram of the Smart Greenhouse

4.2 Pin Connections

Module	Module pin	ESP32 connection	Notes
Soil sensor v1.2	VCC	3.3 V (3V3 pin)	
Soil sensor v1.2	AOUT	GPIO34	
Soil sensor v1.2	GND	GND	
LDR + 10 kΩ	LDR end	3.3 V (3V3 pin)	
LDR + 10 kΩ	Junction	GPIO35	LDR from 3.3 V to GPIO35, 10 kΩ from GPIO35 to GND
LDR + 10 kΩ	10k end	GND	
DHT11	VCC	3.3 V (3V3 pin)	
DHT11	DATA	GPIO4	
DHT11	GND	GND	
2-ch Relay	VCC	5V rail (VIN)	
2-ch Relay	IN1	GPIO26	

Module	Module pin	ESP32 connection	Notes
2-ch Relay	IN2	GPIO27	
2-ch Relay	GND	GND	

Table 3: Pin connection table

4.3 Interfacing and Power Notes

- Power the soil sensor from 3.3 V so its output never exceeds the ADC range.
- Use a separate supply for a 12 V fan or pump; only the relay contacts carry that current.
- Only ADC1 pins (GPIO32-39) are used for analog inputs, because ADC2 cannot be read while Wi-Fi is active.
- Strapping pins GPIO0, 2, 12 and 15 are avoided for inputs so the board always boots normally.
- All grounds are common. 5 V signals are reduced to 3.3 V with resistor dividers before reaching the ESP32.

5. Data Flow Diagrams

Data flow diagrams (DFDs) show how data moves through the system without showing control logic or timing. External entities are drawn as rectangles, processes as numbered circles, data stores as open boxes and data flows as labelled arrows.

5.1 DFD Level 0 (Context Diagram)

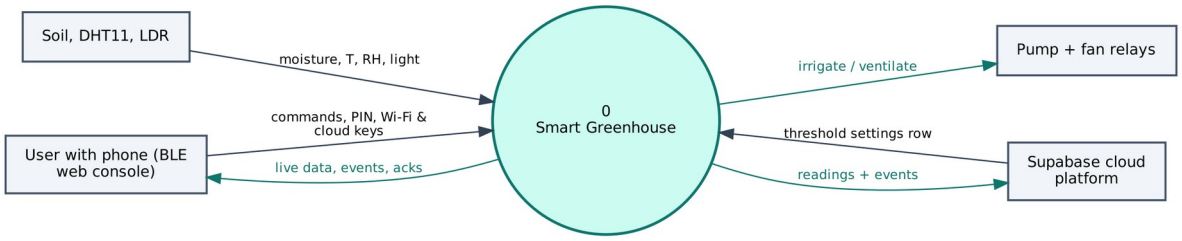


Figure 3: DFD Level 0 — context diagram

At level 0 the whole Smart Greenhouse is a single process. It receives moisture, T, RH, light from the Soil, DHT11, LDR, commands and configuration from the user's phone, and commands from Supabase. It sends irrigate / ventilate to the Pump + fan relays, live data to the phone and readings + events to Supabase.

5.2 DFD Level 1

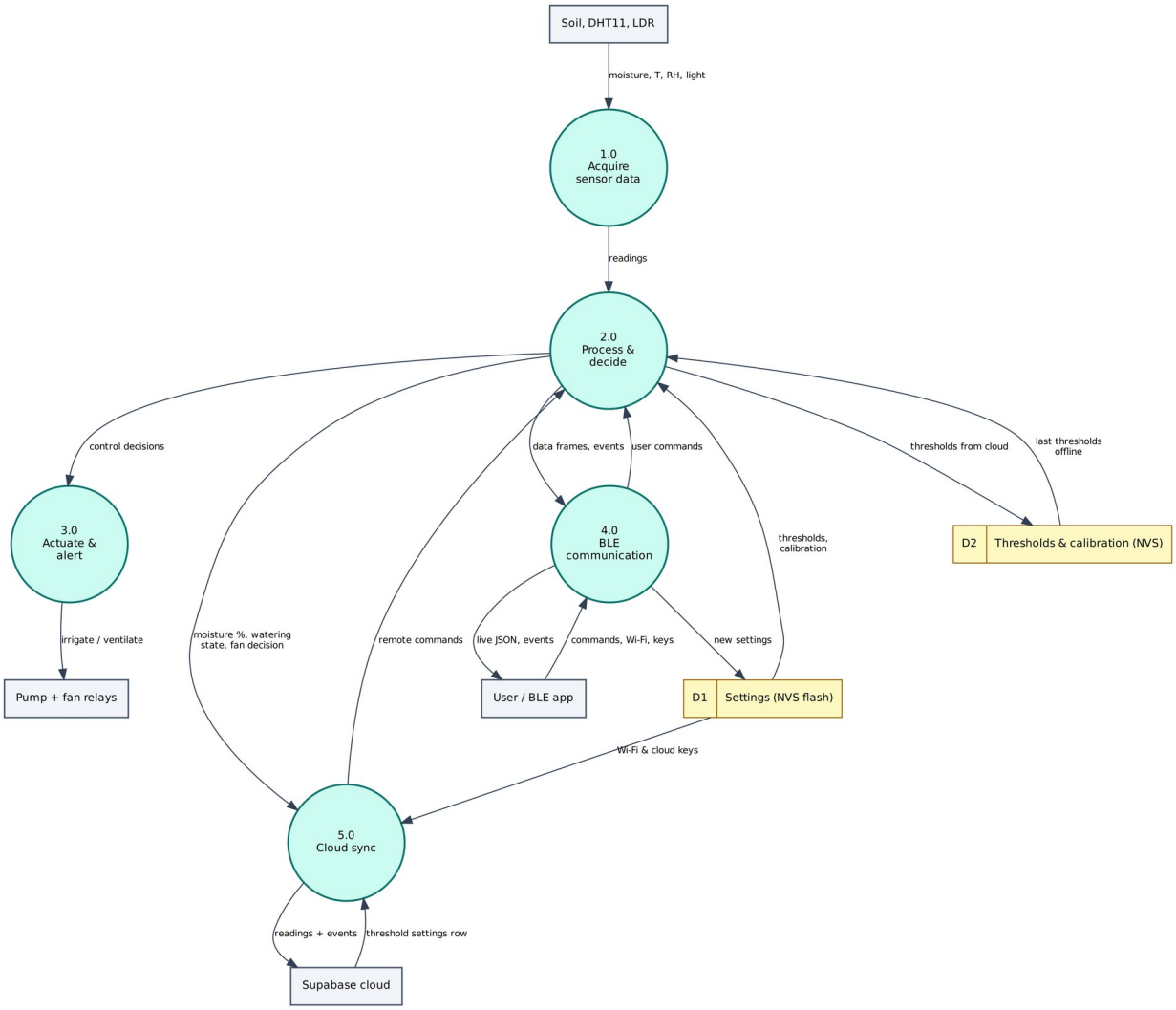


Figure 4: DFD Level 1 — main processes and data stores

Process / store	Description
1.0 Acquire sensor data	Reads moisture, T, RH, light from the Soil, DHT11, LDR; filters/averages the raw values.
2.0 Process & decide	Computes moisture %, watering state, fan decision; applies thresholds and rules; generates events.
3.0 Actuate & alert	Drives the Pump + fan relays (irrigate / ventilate).
4.0 BLE communication	Sends JSON frames and events to the phone; receives and authenticates commands.
5.0 Cloud sync	Sends readings + events to Supabase; receives threshold settings row.
D1 Settings (NVS)	Wi-Fi credentials, cloud keys, PIN, thresholds — survive reboots.
D2 Thresholds & calibration (NVS)	Stores thresholds from cloud; returns last thresholds offline.

Table 4: Description of level-1 processes and data stores

5.3 Data Dictionary

Field	Meaning	Unit
soil	Soil moisture	%
t	Temperature	℃
h	Humidity	%
l	Light	%
pump / fan	Output states	bool
mode	AUTO / MANUAL	—
state	idle / watering / soaking	—
smin, smax, tmax	Active thresholds	% / ℃

Table 5: BLE data frame fields

Cloud object	Content / usage
greenhouse_readings	soil, temperature, humidity, light, pump, fan every 60 s
greenhouse_events	pump_on/off, fan_on/off with reason
greenhouse_settings	Row id=1: soil_min, soil_max, temp_max, hum_max, auto_mode (edit to tune)

Table 6: Supabase data objects

6. Software Design and Implementation

6.1 Development Environment

Item	Details
IDE / build	Arduino IDE 2.x or arduino-cli; GitHub Actions builds firmware in the cloud
Board package	esp32 by Espressif Systems 3.x (tested with 3.3.12)
Board settings	ESP32 Dev Module, Partition Scheme: Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)
Common libraries	NimBLE-Arduino 2.x, ArduinoJson 7.x, PubSubClient 2.8
Project libraries	DHT sensor library (Adafruit), Adafruit Unified Sensor
Phone app	Web Bluetooth console (Chrome on Android / desktop, Bluefy on iPhone); nRF Connect as alternative
Cloud	Supabase (free tier)

Table 7: Software tools

> The default 1.2 MB app partition is too small for BLE + Wi-Fi + TLS. Select **Minimal SPIFFS (1.9MB APP)** in Tools → Partition Scheme, otherwise the upload fails with 'Sketch too big'.

6.2 Firmware Architecture

The firmware is split into three layers so that the ten projects share tested code:

- **IoTCore.h** (shared) — Wi-Fi station with auto-reconnect, NimBLE GATT server with the Nordic UART Service, line-based command parser shared by BLE and USB serial, PIN authentication, settings in NVS (Preferences), NTP time in IST, JSON helpers.
- **Cloud.h** (shared) — ThingSpeak HTTPS client, Adafruit IO MQTT/TLS client with a 28 messages/minute rate limiter, Supabase PostgREST client (insert, update, select).
- **P09_Smart_Greenhouse.ino** — sensor drivers, algorithms, automation rules and the project's own commands.

The main loop is non-blocking: every task is scheduled with millis() timers, BLE writes are queued from the BLE callback into a FreeRTOS queue and executed in loop(), so the BLE stack never waits for slow operations such as HTTPS requests.

6.3 Working of the System

Every 2 s the controller reads all sensors (16-sample ADC averages) and runs the automation. The irrigation state machine has three states. IDLE → WATERING when soil < soil_min; WATERING → IDLE when soil ≥ soil_max, or → SOAKING after a 15 s burst; SOAKING → WATERING after 120 s if soil is still below soil_max, else IDLE.

The fan turns on when temperature > temp_max or humidity > hum_max and off only when temperature < temp_max − 1.5 °C and humidity < hum_max − 5 %. Every pump/fan change is inserted into greenhouse_events with its reason.

Every 60 s a reading row is inserted into greenhouse_readings and the settings row is fetched; changed thresholds are saved in NVS so the greenhouse keeps working with the last values when offline. The

auto_mode column is applied only when it changes, so a manual override from BLE is not undone every minute.

6.4 Flowchart

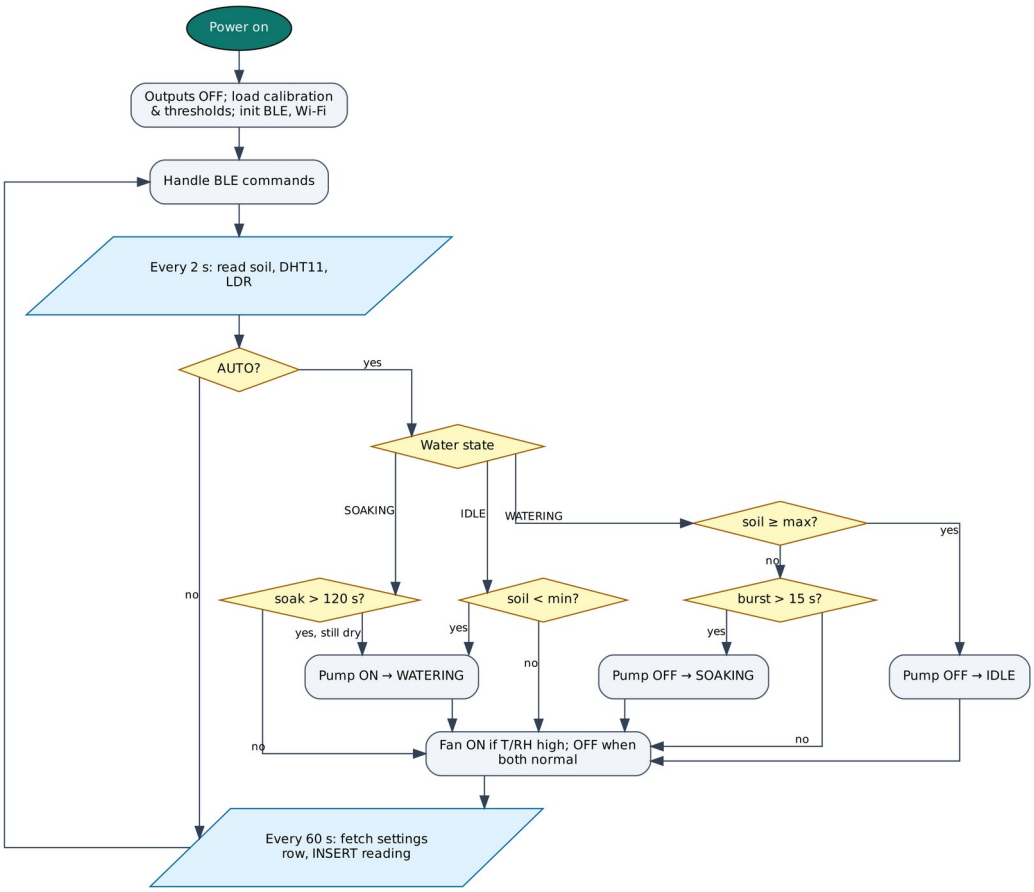


Figure 5: Firmware flowchart

6.5 BLE Communication Protocol

The device is a BLE peripheral named **IOT-P09-XXXX** (XXXX = last bytes of the MAC address). It exposes the Nordic UART Service, which most BLE apps understand:

Item	UUID	Properties
Service (NUS)	6E400001-B5A3-F393-E0A9-E50E24DCCA9E	Primary
RX — phone → ESP32	6E400002-B5A3-F393-E0A9-E50E24DCCA9E	Write, Write without response
TX — ESP32 → phone	6E400003-B5A3-F393-E0A9-E50E24DCCA9E	Notify, Read

Table 8: GATT service and characteristics

Commands are plain text lines (`command:argument`). Replies are one JSON object per line ending in a newline; long lines are split into MTU-sized notifications and re-joined by the app. Commands that change anything require `auth:<PIN>` first on each connection (default PIN 1234, change it with `set:pin=XXXX`).

Command	PIN needed	Action
auth:<PIN>	—	Unlock for this connection
status	No	Status JSON: Wi-Fi, IP, RSSI, heap, uptime, time

Command	PIN needed	Action
wifi:<SSID>,<password>	Yes	Save Wi-Fi credentials and connect
set:<key>=<value>	Yes	Save a setting (cloud keys, thresholds)
get:<key>	Yes	Read a setting (secrets masked)
reboot / forget	Yes	Restart / erase all settings
mode:auto / mode:manual	Yes	Select mode (pump turns off on change)
pump:on|off, fan:on|off	Yes	Manual control (switches to MANUAL)
caldry / calwet	Yes	Store current soil reading as dry / wet reference

Table 9: BLE commands (common commands first, then project commands)

Example data frame sent by the device:

```
{"soil":41,"t":31.2,"h":67,"l":82,"pump":false,"fan":true,"mode":"AUTO","state":"soaking","smin":35,"smax":60,"tmax":32,"type":"data"}
```

6.6 Cloud Integration (Supabase)

Supabase exposes every Postgres table through a REST API (PostgREST). The device sends HTTPS requests to `https://<project>.supabase.co/rest/v1/<table>` with the `apikey` and `Authorization: Bearer` headers: POST to insert rows, PATCH with a filter such as `?id=eq.1` to update and GET with `?select=` to read. Row Level Security policies (in cloud/supabase_schema.sql) allow the device key to do only what the project needs.

Object	Content
greenhouse_readings	soil, temperature, humidity, light, pump, fan every 60 s
greenhouse_events	pump_on/off, fan_on/off with reason
greenhouse_settings	Row id=1: soil_min, soil_max, temp_max, hum_max, auto_mode (edit to tune)

Table 10: Supabase configuration

6.7 Configurable Settings

Key	Default	Meaning
sburl / sbkey	—	Supabase URL and publishable key
soildry / soilwet	2800 / 1200	Soil calibration (mV)
ssid / pass	—	Wi-Fi network (set with the wifi: command)
pin	1234	BLE PIN

Table 11: Settings stored in flash (change with set:key=value)

7. Setup and Deployment Guide

7.1 Hardware Assembly

1. Place the ESP32 DevKit across the centre gap of the breadboard so both pin rows are free.
2. Connect the ESP32 3V3 pin to one breadboard power rail (3.3 V), VIN to the other rail (5 V) and both GND pins to the ground rails.
3. Wire each module exactly as in the pin connection table (section 4.2), including the resistor dividers marked with [n] in the circuit diagram.
4. Check every connection against the circuit diagram before powering on. Make sure no 5 V signal reaches an ESP32 pin directly.
5. Connect the USB cable to a computer (for flashing) or a 5 V adapter (for normal use).

7.2 Flashing the Firmware

Option A — Arduino IDE

1. Install Arduino IDE 2.x from arduino.cc.
2. File → Preferences → Additional boards manager URLs: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`.
3. Tools → Board → Boards Manager: install **esp32 by Espressif Systems** (version 3.x).
4. Sketch → Include Library → Manage Libraries: install **NimBLE-Arduino** (2.x), **ArduinoJson** (7.x), **PubSubClient**, **DHT sensor library (Adafruit)**, **Adafruit Unified Sensor**.
5. Download the repository and open the folder **P09_Smart_Greenhouse** (it already contains IoTCore.h, Cloud.h and config.h).
6. Tools: Board **ESP32 Dev Module**, Partition Scheme **Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)**, Upload Speed 921600, and the correct COM port.
7. Click Upload. If it stays at 'Connecting...', hold the BOOT button on the ESP32 until upload starts.
8. Open Serial Monitor at 115200 baud and type `status` to check the device.

Option B — no software installation (GitHub Actions + browser flasher)

1. Open the repository https://github.com/rohitkukreja705/esp32-iot-projects → **Actions** → latest **Build firmware** run.
2. Download the artifact **firmware-P09_Smart_Greenhouse** and unzip it to get `P09_Smart_Greenhouse.ino.merged.bin`.
3. Open **https://espressif.github.io/esptool-js/** in Chrome or Edge on a computer, click **Connect** and select the ESP32's serial port.
4. Set Flash Address **0x0**, choose the merged .bin file and click **Program**. Press the EN (reset) button when done.

7.3 Cloud Setup (Supabase)

1. Go to **supabase.com**, sign in (GitHub sign-in works) and click **New project**. Choose region **Mumbai (ap-south-1)** for the lowest latency and set a database password.

2. Open **SQL Editor → New query**, paste the whole file `cloud/supabase_schema.sql` from the repository and click **Run**. It creates the tables, views and Row Level Security policies for all four Supabase projects (P05, P06, P09, P10), so one Supabase project can serve all of them.
3. Click **Connect** at the top of the dashboard (or **Project Settings → API Keys**) and copy the **Project URL** (https://xxxx.supabase.co) and the **publishable key** (`sb_publishable_…`; the legacy `anon` key also works).
4. In the BLE web console open **Setup**, unlock and save them in `sburl` and `sbkey`.
5. Open **Table Editor** to watch rows arrive. Use the **SQL Editor** for reports, e.g. the views created by the schema, and **Export to CSV** for spreadsheets.
6. Free tier: 500 MB database and 2 active projects. Free projects are **paused after one week of inactivity** — open the dashboard and click **Restore** if that happens.

Field / feed / table	Content
greenhouse_readings	soil, temperature, humidity, light, pump, fan every 60 s
greenhouse_events	pump_on/off, fan_on/off with reason
greenhouse_settings	Row id=1: soil_min, soil_max, temp_max, hum_max, auto_mode (edit to tune)

Table 12: Items to create in Supabase

7.4 BLE App Setup (Web Bluetooth Console)

No app installation is needed. The console is a web page hosted free on GitHub Pages; Chrome talks to the ESP32 directly through the Web Bluetooth API.

1. On an Android phone turn on **Bluetooth** and **Location** (Android requires location for BLE scanning).
2. Open **https://rohitkukreja705.github.io/esp32-iot-projects/?p=P09** in **Chrome**. (On iPhone use the free **Bluefy** browser; on a laptop use Chrome or Edge.)
3. Tap **Connect** and choose **IOT-P09-XXXX** in the pop-up list. The dot turns green and the Live tab starts updating.
4. Open **Setup**, enter the PIN (default **1234**) and tap **Unlock**.
5. Enter your 2.4 GHz Wi-Fi name and password and tap **Save Wi-Fi**. After a few seconds the status line shows the IP address.
6. Enter the Supabase keys from section 7.3 and tap **Save** next to each.
7. Use the **Control** tab for commands and the **Log** tab to see every event and acknowledgement.

Alternative: nRF Connect app

1. Install **nRF Connect for Mobile** (Nordic Semiconductor, free) and scan.
2. Connect to **IOT-P09-XXXX** and open **Nordic UART Service**.
3. Enable notifications on the **TX** characteristic (6E400003…).
4. Write UTF-8 text to the **RX** characteristic (6E400002…), e.g. `auth:1234`, then `status`.

> To host the console yourself: in the GitHub repository open Settings → Pages, choose 'Deploy from a branch', branch main, folder /docs. Web Bluetooth only works on HTTPS pages, which GitHub Pages provides.

7.5 First-Run Checklist

- Serial Monitor shows '[BLE] advertising as IOT-P09-XXXX'.
- Status in the console shows wifi: true and an IP address.
- Data appears in Supabase within a minute.
- All sensor values in the Live tab are plausible (no '—').

8. Testing and Results

The following test cases verify each function. Record the observed result and mark Pass/Fail while testing your build.

#	Test	Procedure	Expected result	Observed
T1	Calibration	Probe in air: caldry; probe in water: calwet	Air ≈ 0 %, water ≈ 100 %	
T2	Burst-and-soak	Put probe in dry soil (AUTO)	Pump 15 s ON, 120 s OFF, repeats until soil ≥ max	
T3	Fan	Set temp_max 2 °C below room temperature in Supabase	Within 1 min the fan turns ON; 'fan_on' event	
T4	Cloud thresholds	Change soil_min in greenhouse_settings	BLE shows new smin within 60 s	
T5	Manual override	Send pump:on over BLE	Pump ON, mode MANUAL; cloud does not revert it	
T6	Offline	Disconnect Wi-Fi for 10 min	Control continues with last thresholds; logging resumes after	

Table 13: Test cases

8.1 Expected Output

A typical data frame received in the console's Log tab looks like this (values will differ):

```
{"soil":41,"t":31.2,"h":67,"l":82,"pump":false,"fan":true,"mode":"AUTO","state":"soaking","smin":35,"smax":60,"tmax":32,"type":"data"}
```

Serial Monitor (115200 baud) prints the same lines along with connection messages such as `[WiFi] connected`, `[BLE] connected` and the HTTP or MQTT result of each cloud upload, which helps in troubleshooting.

8.2 Troubleshooting

Symptom	Likely cause and fix
Device not in the Connect list	Bluetooth/Location off on the phone, or the page is not opened in Chrome over HTTPS.
'Sketch too big' on upload	Select Partition Scheme 'Minimal SPIFFS (1.9MB APP with OTA)'.
Wi-Fi never connects	ESP32 supports only 2.4 GHz networks; check the SSID/password (case-sensitive).
Settings refused	Send auth:<PIN> first; the unlock is per BLE connection.
Sensor shows null / —	Check wiring and supply voltage; see the Serial Monitor for 'not found' messages.
Board resets randomly	Weak USB supply or servo/relay current; use a 5 V 2 A adapter and add a 470 µF capacitor.
No data in Supabase	Wrong URL/key, schema not run, or project paused after inactivity.

Table 14: Troubleshooting guide

9. Discussion

9.1 Advantages

- Saves water through burst-and-soak irrigation.
- Corrosion-free capacitive sensing.
- Thresholds changeable from anywhere without reflashing.
- Full history of readings and actions in a SQL database.

9.2 Limitations

- One moisture sensor represents the whole zone.
- DHT11 has limited accuracy and range (0-50 °C).
- Capacitive sensor readings vary with soil type; calibrate in the actual soil for best results.
- Supabase free projects pause after a week without activity.

9.3 Safety Precautions

- Keep electronics in an IP65 box; route pump and fan wiring with drip loops.
- Use low-voltage pumps/fans where possible; mains pumps need a contactor and RCCB.

9.4 Security Considerations

- Secrets (Wi-Fi password, cloud keys) are stored in the ESP32's NVS flash, not in source code.
- BLE commands that change the device require a PIN; change the default PIN after setup.
- Cloud traffic is TLS-encrypted. Certificate verification is disabled (setInsecure) for simplicity; for production, pin the server's root certificate with setCACert().
- Use a dedicated cloud account/project for the device and rotate keys if the device is lost.

9.5 Future Scope

- Multiple zones with solenoid valves and one sensor per zone.
- Use weather forecast (rain probability) to skip irrigation.
- Shade-net motor control using the LDR reading.
- Web dashboard with charts using Supabase JS client.

10. Conclusion

The Smart Greenhouse automates irrigation and ventilation using inexpensive sensors and a robust control strategy. Burst-and-soak watering avoids the overshoot of simple threshold controllers, and storing thresholds in Supabase lets growers tune the system remotely, while BLE and offline thresholds keep it working without internet.

All design goals listed in Chapter 1 are met with low-cost parts available in the Indian market, free cloud services and open-source software. The shared firmware core and the single Web Bluetooth console make the project easy to reproduce and extend.

References

1. DFRobot, Capacitive Soil Moisture Sensor v1.2 (SEN0193) wiki.
2. FAO Irrigation and Drainage Paper 56 — Crop evapotranspiration (background on irrigation scheduling).
3. Aosong, DHT11 datasheet.
4. Supabase documentation — Database, Table Editor and REST API.
5. Espressif Systems, ESP32 Series Datasheet and ESP32 Technical Reference Manual, espressif.com.
6. Espressif Systems, Arduino-ESP32 documentation, docs.espressif.com/projects/arduino-esp32.
7. H2zero, NimBLE-Arduino library documentation, github.com/h2zero/NimBLE-Arduino.
8. Bluetooth SIG, Bluetooth Core Specification 4.2 — Generic Attribute Profile (GATT).
9. W3C Web Bluetooth Community Group, Web Bluetooth specification, webbluetoothcg.github.io/web-bluetooth.
10. B. Blanchon, ArduinoJson 7 documentation, arduinojson.org.
11. Supabase Inc., Supabase documentation — REST API and Row Level Security, supabase.com/docs.

Appendix A: Project Source Code

The complete code is in the repository folder **P09_Smart_Greenhouse** (https://github.com/rohitkukreja705/esp32-iot-projects).

config.h

```
#pragma once
// ===== P09 Smart Greenhouse / Irrigation — configuration =====
#define WIFI_SSID     ""
#define WIFI_PASS     ""
#define DEVICE_PIN    "1234"
#define SUPABASE_URL  ""
#define SUPABASE_KEY  ""

// ----- pins -----
#define PIN_SOIL      34     // capacitive soil moisture v1.2 AOUT (powered 3.3 V)
#define PIN_LDR       35     // LDR divider
#define PIN_DHT       4      // DHT11
#define PIN_PUMP      26     // relay CH1 -> 5 V mini pump
#define PIN_FAN       27     // relay CH2 -> 5 V/12 V fan
#define RELAY_ACTIVE_LOW true

// ----- defaults; the live values come from the Supabase "greenhouse_settings" row -----
#define DEF_SOIL_MIN   35    // % : start watering below
#define DEF_SOIL_MAX   60    // % : stop watering at
#define DEF_TEMP_MAX   32    // °C: fan on above
#define DEF_HUM_MAX    85    // %RH: fan on above
#define DEF_SOIL_DRY_MV 2800 // sensor in dry air   (set:soildry=)
#define DEF_SOIL_WET_MV 1200 // sensor in water     (set:soilwet=)
#define PUMP_BURST_S   15    // water in short bursts...
#define SOAK_S         120   // ...then wait for water to spread before re-checking

#define SAMPLE_MS      2000
#define UPLOAD_MS      60000
#define SETTINGS_MS    60000
```

P09_Smart_Greenhouse.ino

```
/*
 * P09 — Smart Greenhouse with Cloud-Configurable Irrigation
 * ============================================================
 * Peripherals: capacitive soil-moisture sensor v1.2, DHT11, LDR,
 *              2-channel relay (water pump + ventilation fan)
 * Wi-Fi  : Supabase — readings every minute, events log, and thresholds read
 *          from a settings row (change them in the Supabase table editor and
 *          the greenhouse follows within a minute)
 * BLE    : live dashboard, manual pump/fan, AUTO/MANUAL mode, soil calibration
 * Logic  : burst-and-soak watering (prevents over-watering), fan control by
 *          temperature OR humidity with hysteresis, works fully offline with
 *          the last known thresholds
 */
#include "config.h"
#include "IoTCore.h"
#include "Cloud.h"
#include <DHT.h>

DHT dht(PIN_DHT, DHT11);

struct Th { float soilMin, soilMax, tempMax, humMax; } th = { DEF_SOIL_MIN, DEF_SOIL_MAX, DEF_TEMP_MAX,
DEF_HUM_MAX };
```

```
float soil = NAN, temp = NAN, hum = NAN, light = NAN;
bool pumpOn = false, fanOn = false, autoMode = true;
enum WaterState { IDLE, WATERING, SOAKING } ws = IDLE;
uint32_t stateSince = 0, lastSample = 0, lastUpload = 0, lastSettings = 0;
float dryMv, wetMv;
int pumpCyclesToday = 0;

void setOut(int pin, bool on) { digitalWrite(pin, (on ^ RELAY_ACTIVE_LOW) ? HIGH : LOW); }

void logEvent(const String& ev, const String& detail) {
  iot::sendEvent(ev, detail);
  if (!iot::wifiOK()) return;
  JsonDocument d; d["event"] = ev; d["detail"] = detail; d["device"] = iot::deviceName;
  String s; serializeJson(d, s);
  cloud::supabaseInsert("greenhouse_events", s);
}

void setPump(bool on, const String& why) {
  if (on == pumpOn) return;
  pumpOn = on; setOut(PIN_PUMP, on);
  logEvent(on ? "pump_on" : "pump_off", why);
}
void setFan(bool on, const String& why) {
  if (on == fanOn) return;
  fanOn = on; setOut(PIN_FAN, on);
  logEvent(on ? "fan_on" : "fan_off", why);
}

float avgMv(int pin) {
  uint32_t s = 0; for (int i = 0; i < 16; i++) { s += analogReadMilliVolts(pin); delayMicroseconds(250);
}
  return s / 16.0;
}

void sample() {
  float t = dht.readTemperature(), h = dht.readHumidity();
  if (!isnan(t)) temp = t;
  if (!isnan(h)) hum = h;
  // capacitive sensor: lower voltage = wetter soil
  soil = constrain((dryMv - avgMv(PIN_SOIL)) / (dryMv - wetMv) * 100.0, 0, 100);
  light = constrain(avgMv(PIN_LDR) / 3300.0 * 100.0, 0, 100);
}

void automation() {
  if (!autoMode || isnan(soil)) return;
  uint32_t now = millis(), inState = now - stateSince;
  switch (ws) {
    case IDLE:
      if (soil < th.soilMin) { ws = WATERING; stateSince = now; setPump(true, "soil " + String(soil, 0)
+ "% < " + String(th.soilMin, 0) + "%"); pumpCyclesToday++; }
      break;
    case WATERING:
      if (soil >= th.soilMax) { setPump(false, "target reached"); ws = IDLE; stateSince = now; }
      else if (inState > PUMP_BURST_S * 1000UL) { setPump(false, "burst done, soaking"); ws = SOAKING;
stateSince = now; }
      break;
    case SOAKING:
      if (inState > SOAK_S * 1000UL) {
        if (soil < th.soilMax) { ws = WATERING; stateSince = now; setPump(true, "still dry after soak");
}
        else { ws = IDLE; stateSince = now; }
      }
      break;
```

```
  }
  if (!isnan(temp) && !isnan(hum)) {
    bool hot = temp > th.tempMax || hum > th.humMax;
    bool cool = temp < th.tempMax - 1.5 && hum < th.humMax - 5;   // hysteresis
    if (hot && !fanOn) setFan(true, "T " + String(temp, 1) + "C / RH " + String(hum, 0) + "%");
    if (cool && fanOn) setFan(false, "climate normal");
  }
}

void fetchCloudSettings() {
  String out;
  if (!cloud::supabaseSelect("greenhouse_settings", "select=*&id=eq.1", out)) return;
  JsonDocument d;
  if (deserializeJson(d, out) || !d.is<JsonArray>() || d.size() == 0) return;
  JsonObject o = d[0];
  Th n = { o["soil_min"] | th.soilMin, o["soil_max"] | th.soilMax, o["temp_max"] | th.tempMax,
o["hum_max"] | th.humMax };
  if (n.soilMin != th.soilMin || n.soilMax != th.soilMax || n.tempMax != th.tempMax || n.humMax !=
th.humMax) {
    th = n;
    iot::saveSetting("th", String(th.soilMin) + "," + String(th.soilMax) + "," + String(th.tempMax) +
"," + String(th.humMax));
    iot::sendEvent("thresholds", "Updated from cloud");
  }
  // apply the cloud mode only when it is changed in the table, so a manual
  // override made over BLE is not undone every minute
  static int lastCloudAuto = -1;
  if (!o["auto_mode"].isNull()) {
    int a = o["auto_mode"].as<bool>() ? 1 : 0;
    if (a != lastCloudAuto) { if (lastCloudAuto != -1 || a == 0) autoMode = a; lastCloudAuto = a; }
  }
}

void upload() {
  JsonDocument d;
  d["soil"] = iot::rnd(soil, 0); d["temperature"] = iot::rnd(temp); d["humidity"] = iot::rnd(hum, 0);
  d["light"] = iot::rnd(light, 0); d["pump"] = pumpOn; d["fan"] = fanOn; d["device"] = iot::deviceName;
  String s; serializeJson(d, s);
  cloud::supabaseInsert("greenhouse_readings", s);
}

void sendBle() {
  JsonDocument d;
  d["soil"] = iot::rnd(soil, 0); d["t"] = iot::rnd(temp); d["h"] = iot::rnd(hum, 0); d["l"] =
iot::rnd(light, 0);
  d["pump"] = pumpOn; d["fan"] = fanOn; d["mode"] = autoMode ? "AUTO" : "MANUAL";
  d["state"] = ws == IDLE ? "idle" : ws == WATERING ? "watering" : "soaking";
  d["smin"] = th.soilMin; d["smax"] = th.soilMax; d["tmax"] = th.tempMax;
  iot::sendData(d);
}

void loadLocal() {
  dryMv = iot::settingF("soildry", DEF_SOIL_DRY_MV);
  wetMv = iot::settingF("soilwet", DEF_SOIL_WET_MV);
  String s = iot::setting("th", "");
  if (s.length()) sscanf(s.c_str(), "%f,%f,%f,%f", &th.soilMin, &th.soilMax, &th.tempMax, &th.humMax);
}

// Commands: pump:on|off  fan:on|off  mode:auto|manual  caldry  calwet
bool onCommand(const String& cmd, const String& arg, bool authed) {
  if (cmd == "setting") { loadLocal(); return true; }
  if (cmd == "pump" || cmd == "fan" || cmd == "mode" || cmd == "caldry" || cmd == "calwet") {
    if (!authed) { iot::emitAck(cmd, false, "send auth:<PIN> first"); return true; }
```

```
    if (cmd == "mode") { autoMode = arg == "auto"; ws = IDLE; setPump(false, "mode change"); }
    if (cmd == "pump") { autoMode = false; setPump(arg == "on", "manual (BLE)"); }
    if (cmd == "fan")  { autoMode = false; setFan(arg == "on", "manual (BLE)"); }
    if (cmd == "caldry") { dryMv = avgMv(PIN_SOIL); iot::saveSetting("soildry", String(dryMv, 0)); }
    if (cmd == "calwet") { wetMv = avgMv(PIN_SOIL); iot::saveSetting("soilwet", String(wetMv, 0)); }
    iot::emitAck(cmd, true, autoMode ? "AUTO" : "MANUAL");
    sendBle();
    return true;
  }
  return false;
}

void setup() {
  pinMode(PIN_PUMP, OUTPUT); pinMode(PIN_FAN, OUTPUT);
  setOut(PIN_PUMP, false); setOut(PIN_FAN, false);
  analogSetAttenuation(ADC_11db);
  iot::begin("P09", onCommand);
  loadLocal();
  dht.begin();
}

void loop() {
  iot::loop();
  uint32_t now = millis();
  if (now - lastSample >= SAMPLE_MS) { lastSample = now; sample(); automation(); sendBle(); }
  if (iot::wifiOK() && (lastSettings == 0 || now - lastSettings >= SETTINGS_MS)) { lastSettings = now;
fetchCloudSettings(); }
  if (iot::wifiOK() && now - lastUpload >= UPLOAD_MS && !isnan(soil)) { lastUpload = now; upload(); }
}
```

Appendix B: Shared Library Code

These two headers are identical in every project folder (master copies in /common).

IoTCore.h

```
/*
 * IoTCore.h  —  shared runtime for all ESP32 IoT projects in this repo
 * ---------------------------------------------------------------------
 *  - Wi-Fi station with automatic reconnect (credentials stored in flash)
 *  - BLE server using the Nordic UART Service (NUS) layout, so the device
 *    works with the Web Bluetooth dashboard in /docs AND with generic apps
 *    such as "nRF Connect" or "Serial Bluetooth Terminal"
 *  - Line-based command protocol shared by BLE and USB Serial
 *  - Persistent settings (Preferences / NVS): Wi-Fi, cloud keys, PIN, thresholds
 *  - NTP time sync (IST) for timestamps
 *
 *  Wire protocol
 *  -------------
 *  Phone -> ESP32 (write to RX characteristic, text):
 *      auth:1234              unlock control + settings for this connection
 *      status                 device replies with a status JSON line
 *      set:key=value          store a setting (Wi-Fi, cloud key, threshold…)
 *      get:key                read back a setting (secrets are masked)
 *      wifi:SSID,PASSWORD     shortcut for set:ssid + set:pass + reconnect
 *      reboot                 restart the board
 *      <anything else>        forwarded to the project's own command handler
 *  ESP32 -> Phone (notify on TX characteristic):
 *      one JSON object per line, terminated by '\n'
 *      {"type":"data",...}  {"type":"status",...}  {"type":"ack",...}
 *      {"type":"event",...} {"type":"err",...}
 *  Lines longer than one BLE packet are split into chunks; the receiver
 *  joins chunks until it sees '\n'.
 *
 *  Library: NimBLE-Arduino 2.x (lighter than Bluedroid, leaves more RAM for TLS)
 *  Board:   ESP32 Dev Module, Partition Scheme "Minimal SPIFFS (1.9MB APP with OTA)"
 */
#pragma once

#include <Arduino.h>
#include <WiFi.h>
#include <Preferences.h>
#include <NimBLEDevice.h>
#include <ArduinoJson.h>
#include <time.h>

#ifndef DEVICE_PIN
#define DEVICE_PIN "1234"
#endif
#ifndef WIFI_SSID
#define WIFI_SSID ""
#endif
#ifndef WIFI_PASS
#define WIFI_PASS ""
#endif

namespace iot {

// Nordic UART Service UUIDs (widely supported by BLE terminal apps)
static const char* NUS_SERVICE = "6E400001-B5A3-F393-E0A9-E50E24DCCA9E";
static const char* NUS_RX      = "6E400002-B5A3-F393-E0A9-E50E24DCCA9E";  // phone writes here
static const char* NUS_TX      = "6E400003-B5A3-F393-E0A9-E50E24DCCA9E";  // ESP32 notifies here
```

```
// Project command handler: return true if the command was recognised.
using CommandHandler = bool (*)(const String& cmd, const String& arg, bool authed);

// ---------------------------------------------------------------- state
struct CmdMsg { char text[256]; bool fromBle; };

static Preferences        prefs;
static NimBLEServer*      bleServer   = nullptr;
static NimBLECharacteristic* txChr    = nullptr;
static QueueHandle_t      cmdQueue    = nullptr;
static CommandHandler     userHandler = nullptr;
static String             deviceName;
static String             projectId;
static volatile uint16_t  bleMtu      = 23;
static volatile bool      bleAuthed   = false;   // reset on every new connection
static bool               serialAuthed = true;   // USB = physical access, trusted
static bool               timeSynced  = false;
static uint32_t           lastWifiTry = 0;
static String             rxLine;                // assembles multi-packet writes

// ---------------------------------------------------------------- settings
inline String setting(const char* key, const char* def = "") {
  return prefs.isKey(key) ? prefs.getString(key, def) : String(def);
}
inline void saveSetting(const char* key, const String& v) { prefs.putString(key, v); }
inline float settingF(const char* key, float def) {
  String s = setting(key, "");
  return s.length() ? s.toFloat() : def;
}

// ---------------------------------------------------------------- BLE
inline bool bleConnected() { return bleServer && bleServer->getConnectedCount() > 0; }

// Send one line (adds '\n'), split to fit the negotiated MTU.
inline void bleSend(const String& line) {
  if (!bleConnected() || !txChr) return;
  String msg = line + "\n";
  size_t chunk = (bleMtu > 23 ? bleMtu : 23) - 3;
  for (size_t i = 0; i < msg.length(); i += chunk) {
    size_t n = min(chunk, msg.length() - i);
    txChr->setValue((const uint8_t*)msg.c_str() + i, n);
    txChr->notify();
    if (msg.length() > chunk) delay(8);   // give the stack time between chunks
  }
}

// Send to BLE and mirror to USB Serial.
inline void emit(const String& line) {
  Serial.println(line);
  bleSend(line);
}

inline void emitAck(const String& cmd, bool ok, const String& msg = "") {
  JsonDocument d;
  d["type"] = ok ? "ack" : "err";
  d["cmd"] = cmd;
  if (msg.length()) d["msg"] = msg;
  String s; serializeJson(d, s); emit(s);
}

inline void enqueue(const String& s, bool fromBle) {
  if (!cmdQueue || s.length() == 0) return;
```

```
  CmdMsg m; memset(&m, 0, sizeof(m));
  strncpy(m.text, s.c_str(), sizeof(m.text) - 1);
  m.fromBle = fromBle;
  xQueueSend(cmdQueue, &m, 0);
}

class ServerCB : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer* s, NimBLEConnInfo& info) override {
    bleAuthed = false;
    rxLine = "";
    Serial.printf("[BLE] connected %s\n", info.getAddress().toString().c_str());
  }
  void onDisconnect(NimBLEServer* s, NimBLEConnInfo& info, int reason) override {
    bleAuthed = false;
    bleMtu = 23;
    Serial.printf("[BLE] disconnected (reason %d), advertising again\n", reason);
    // NimBLE restarts advertising automatically (advertiseOnDisconnect = true)
  }
  void onMTUChange(uint16_t mtu, NimBLEConnInfo& info) override {
    bleMtu = mtu;
    Serial.printf("[BLE] MTU %u\n", mtu);
  }
};

class RxCB : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* c, NimBLEConnInfo& info) override {
    std::string v = c->getValue();
    // Accept either a single write without newline, or several writes
    // that together form lines terminated by '\n'.
    for (char ch : v) {
      if (ch == '\r') continue;
      if (ch == '\n') { enqueue(rxLine, true); rxLine = ""; }
      else if (rxLine.length() < 250) rxLine += ch;
    }
    if (v.find('\n') == std::string::npos && rxLine.length() && rxLine.length() == v.length()) {
      enqueue(rxLine, true);  // plain one-shot write (e.g. typed in nRF Connect)
      rxLine = "";
    }
  }
};

inline void bleBegin() {
  uint8_t mac[6]; WiFi.macAddress(mac);
  char suffix[6]; snprintf(suffix, sizeof(suffix), "%02X%02X", mac[4], mac[5]);
  deviceName = "IOT-" + projectId + "-" + suffix;

  NimBLEDevice::init(deviceName.c_str());
  NimBLEDevice::setMTU(247);
  bleServer = NimBLEDevice::createServer();
  bleServer->setCallbacks(new ServerCB());

  NimBLEService* svc = bleServer->createService(NUS_SERVICE);
  txChr = svc->createCharacteristic(NUS_TX, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ);
  NimBLECharacteristic* rx = svc->createCharacteristic(
      NUS_RX, NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR, 512);
  rx->setCallbacks(new RxCB());
  // (NimBLE 2.x starts services automatically when advertising begins)

  // 128-bit UUID goes in the advertisement, the name in the scan response
  // (both together would not fit in one 31-byte packet).
  NimBLEAdvertisementData adv, scan;
  adv.setFlags(BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP);
  adv.setCompleteServices(NimBLEUUID(NUS_SERVICE));
```

```
  scan.setName(deviceName.c_str());
  NimBLEAdvertising* a = NimBLEDevice::getAdvertising();
  a->setAdvertisementData(adv);
  a->setScanResponseData(scan);
  a->start();
  Serial.printf("[BLE] advertising as %s\n", deviceName.c_str());
}

// ---------------------------------------------------------------- Wi-Fi
inline bool wifiOK() { return WiFi.status() == WL_CONNECTED; }

inline void wifiConnect() {
  String ssid = setting("ssid", WIFI_SSID);
  String pass = setting("pass", WIFI_PASS);
  if (ssid.isEmpty()) {
    Serial.println("[WiFi] no SSID yet - send  wifi:SSID,PASSWORD  over BLE");
    return;
  }
  Serial.printf("[WiFi] connecting to %s\n", ssid.c_str());
  WiFi.disconnect();
  WiFi.begin(ssid.c_str(), pass.c_str());
  lastWifiTry = millis();
}

inline void wifiLoop() {
  static bool wasUp = false;
  bool up = wifiOK();
  if (up && !wasUp) {
    Serial.printf("[WiFi] connected, IP %s, RSSI %d\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
    configTzTime("IST-5:30", "pool.ntp.org", "time.google.com");
  }
  if (!up && millis() - lastWifiTry > 20000) wifiConnect();   // retry every 20 s
  wasUp = up;
  if (up && !timeSynced && time(nullptr) > 1700000000) {
    timeSynced = true;
    Serial.println("[NTP] time synced");
  }
}

// ISO-8601 local time, or uptime if NTP has not synced yet.
inline String timestamp() {
  time_t now = time(nullptr);
  if (now < 1700000000) return "uptime+" + String(millis() / 1000) + "s";
  struct tm t; localtime_r(&now, &t);
  char buf[32]; strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S+05:30", &t);
  return String(buf);
}

// ------------------------------------------------------- built-in commands
inline bool isSecretKey(const String& k) {
  return k == "pass" || k.endsWith("key") || k == "pin";
}

inline void sendStatus() {
  JsonDocument d;
  d["type"] = "status";
  d["dev"] = deviceName;
  d["wifi"] = wifiOK();
  d["ssid"] = setting("ssid", WIFI_SSID);
  d["rssi"] = wifiOK() ? WiFi.RSSI() : 0;
  d["ip"] = wifiOK() ? WiFi.localIP().toString() : "";
  d["auth"] = bleAuthed;
```

```
  d["heap"] = ESP.getFreeHeap();
  d["up"] = millis() / 1000;
  d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

inline void handleLine(const String& raw, bool fromBle) {
  String line = raw; line.trim();
  if (line.isEmpty()) return;
  int c = line.indexOf(':');
  String cmd = (c < 0 ? line : line.substring(0, c));
  String arg = (c < 0 ? "" : line.substring(c + 1));
  cmd.trim(); cmd.toLowerCase();
  bool authed = fromBle ? bleAuthed : serialAuthed;

  if (cmd == "auth") {
    bool ok = arg == setting("pin", DEVICE_PIN);
    if (fromBle) bleAuthed = ok;
    emitAck(cmd, ok, ok ? "unlocked" : "wrong PIN");
    return;
  }
  if (cmd == "status") { sendStatus(); return; }
  if (cmd == "ping")   { emitAck(cmd, true, "pong"); return; }

  // Everything below changes the device, so it needs the PIN over BLE.
  if (cmd == "set" || cmd == "wifi" || cmd == "reboot" || cmd == "get" || cmd == "forget") {
    if (!authed) { emitAck(cmd, false, "send auth:<PIN> first"); return; }
    if (cmd == "set") {
      int e = arg.indexOf('=');
      if (e <= 0) { emitAck(cmd, false, "use set:key=value"); return; }
      String k = arg.substring(0, e); k.trim();
      String v = arg.substring(e + 1); v.trim();
      if (k.length() > 15) { emitAck(cmd, false, "key too long"); return; }
      saveSetting(k.c_str(), v);
      emitAck(cmd, true, k + " saved");
      if (k == "ssid" || k == "pass") wifiConnect();
      // let the project react to changed thresholds etc.
      if (userHandler) userHandler("setting", k, true);
      return;
    }
    if (cmd == "get") {
      String v = setting(arg.c_str(), "");
      if (isSecretKey(arg) && v.length()) v = "****" + v.substring(max(0, (int)v.length() - 3));
      emitAck(cmd, true, arg + "=" + v);
      return;
    }
    if (cmd == "wifi") {
      int k = arg.indexOf(',');
      String ssid = k < 0 ? arg : arg.substring(0, k);
      String pass = k < 0 ? "" : arg.substring(k + 1);
      saveSetting("ssid", ssid); saveSetting("pass", pass);
      emitAck(cmd, true, "connecting to " + ssid);
      wifiConnect();
      return;
    }
    if (cmd == "forget") { prefs.clear(); emitAck(cmd, true, "all settings erased"); return; }
    if (cmd == "reboot") { emitAck(cmd, true, "rebooting"); delay(300); ESP.restart(); }
  }

  if (userHandler && userHandler(cmd, arg, authed)) return;
  emitAck(cmd, false, "unknown command");
}
```

```
// ------------------------------------------------------------ entry points
// Call once from setup().  id = short project id, e.g. "P01".
inline void begin(const char* id, CommandHandler handler) {
  Serial.begin(115200);
  delay(200);
  projectId = id;
  userHandler = handler;
  prefs.begin("iot", false);
  cmdQueue = xQueueCreate(8, sizeof(CmdMsg));
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  // NOTE: Wi-Fi modem sleep must stay ON while BLE is running (ESP32 coexistence).
  bleBegin();
  wifiConnect();
  Serial.println("[CORE] ready - type 'status' here or connect over BLE");
}

// Call every loop() iteration.
inline void loop() {
  wifiLoop();
  // USB serial commands (handy while developing)
  static String serialLine;
  while (Serial.available()) {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r') { if (serialLine.length()) enqueue(serialLine, false); serialLine =
""; }
    else if (serialLine.length() < 250) serialLine += ch;
  }
  CmdMsg m;
  while (cmdQueue && xQueueReceive(cmdQueue, &m, 0) == pdTRUE) handleLine(String(m.text), m.fromBle);
}

// Round for compact JSON. NaN stays NaN, which ArduinoJson writes as null.
inline float rnd(float v, int dp = 1) {
  if (isnan(v)) return v;
  float m = powf(10, dp);
  return roundf(v * m) / m;
}

// Helper: build and send a {"type":"data",...} line from a filled document.
inline void sendData(JsonDocument& d) {
  d["type"] = "data";
  String s; serializeJson(d, s); emit(s);
}

inline void sendEvent(const String& name, const String& msg) {
  JsonDocument d;
  d["type"] = "event"; d["name"] = name; d["msg"] = msg; d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

}  // namespace iot
```

Cloud.h

```
/*
 * Cloud.h  –  free-tier cloud helpers used by the projects
 * ---------------------------------------------------------
 *  ThingSpeak   : HTTPS REST "update" (free tier: 1 update / 15 s per channel)
 *  Adafruit IO  : MQTT over TLS (free tier: 30 data points / minute, 10 feeds)
 *  Supabase     : PostgREST over HTTPS (free tier: 500 MB Postgres)
 *
```

```
 *  Keys are read from settings (stored over BLE with set:key=value) so no
 *  secret has to be hard-coded or committed to GitHub:
 *     tskey    ThingSpeak Write API Key
 *     aiouser  Adafruit IO username        aiokey  Adafruit IO key
 *     sburl    https://<ref>.supabase.co   sbkey   Supabase publishable/anon key
 *
 *  TLS note: certificate checking is disabled with setInsecure() to keep the
 *  projects beginner-friendly and independent of CA rotations. Traffic is
 *  still encrypted. For production, pin the root CA with setCACert().
 */
#pragma once

#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <PubSubClient.h>
#include "IoTCore.h"

// Optional compile-time defaults (normally left empty and set over BLE)
#ifndef TS_WRITE_KEY
#define TS_WRITE_KEY ""
#endif
#ifndef AIO_USERNAME
#define AIO_USERNAME ""
#endif
#ifndef AIO_KEY
#define AIO_KEY ""
#endif
#ifndef SUPABASE_URL
#define SUPABASE_URL ""
#endif
#ifndef SUPABASE_KEY
#define SUPABASE_KEY ""
#endif

namespace cloud {

// ============================================================== ThingSpeak
// vals[i] -> field(i+1). Pass NAN to skip a field. Returns true on success.
inline bool thingSpeakUpdate(const float* vals, int n, const String& status = "") {
  if (!iot::wifiOK()) return false;
  String key = iot::setting("tskey", TS_WRITE_KEY);
  if (key.isEmpty()) { Serial.println("[TS] no tskey set"); return false; }
  String url = "https://api.thingspeak.com/update?api_key=" + key;
  for (int i = 0; i < n && i < 8; i++)
    if (!isnan(vals[i])) url += "&field" + String(i + 1) + "=" + String(vals[i], 2);
  if (status.length()) {
    String st = status; st.replace(" ", "%20");
    url += "&status=" + st;
  }
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, url)) return false;
  int code = http.GET();
  String body = http.getString();
  http.end();
  bool ok = code == 200 && body.toInt() > 0;   // ThingSpeak returns the entry id, 0 = rejected
  Serial.printf("[TS] HTTP %d entry=%s\n", code, body.c_str());
  return ok;
}

// ============================================================== Supabase
// Generic PostgREST call. method: GET/POST/PATCH. path e.g. "/rest/v1/readings"
inline int supabaseRequest(const char* method, const String& path, const String& body,
```

```
                           String* response = nullptr, const char* prefer = "return=minimal") {
  if (!iot::wifiOK()) return -1;
  String base = iot::setting("sburl", SUPABASE_URL);
  String key  = iot::setting("sbkey", SUPABASE_KEY);
  if (base.isEmpty() || key.isEmpty()) { Serial.println("[SB] sburl/sbkey not set"); return -2; }
  if (base.endsWith("/")) base.remove(base.length() - 1);
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, base + path)) return -3;
  http.addHeader("apikey", key);
  http.addHeader("Authorization", "Bearer " + key);
  http.addHeader("Content-Type", "application/json");
  if (prefer) http.addHeader("Prefer", prefer);
  int code = http.sendRequest(method, body);
  if (response) *response = http.getString();
  else if (code >= 300) Serial.printf("[SB] %s\n", http.getString().c_str());
  http.end();
  Serial.printf("[SB] %s %s -> %d\n", method, path.c_str(), code);
  return code;
}

inline bool supabaseInsert(const char* table, const String& json) {
  int c = supabaseRequest("POST", String("/rest/v1/") + table, json);
  return c >= 200 && c < 300;
}

// filter is a PostgREST filter, e.g. "id=eq.1"
inline bool supabaseUpdate(const char* table, const String& filter, const String& json) {
  int c = supabaseRequest("PATCH", String("/rest/v1/") + table + "?" + filter, json);
  return c >= 200 && c < 300;
}

// query e.g. "select=*&id=eq.1"  -> JSON array text in out
inline bool supabaseSelect(const char* table, const String& query, String& out) {
  int c = supabaseRequest("GET", String("/rest/v1/") + table + "?" + query, "", &out, nullptr);
  return c == 200;
}

// ============================================================ Adafruit IO
// MQTT wrapper with a simple rate limiter to respect the 30 msg/min limit.
using FeedCallback = void (*)(const String& feed, const String& value);

class AdafruitIO {
 public:
  // requestLast = true asks Adafruit IO for each feed's last value on connect
  // (useful for switches changed while offline; unsafe for things like pumps)
  void begin(FeedCallback cb, const char* const* subFeeds, int nSub, bool requestLast = true) {
    cb_ = cb; subs_ = subFeeds; nSub_ = nSub; requestLast_ = requestLast;
    tls_.setInsecure();
    mqtt_.setClient(tls_);
    mqtt_.setServer("io.adafruit.com", 8883);
    mqtt_.setBufferSize(512);
    mqtt_.setKeepAlive(60);
    mqtt_.setCallback([](char* topic, byte* payload, unsigned int len) {
      String t(topic), v;
      for (unsigned int i = 0; i < len; i++) v += (char)payload[i];
      int p = t.lastIndexOf('/');
      String feed = t.substring(p + 1);
      if (instance()->cb_) instance()->cb_(feed, v);
    });
    instance() = this;
  }
```

```
  void loop() {
    if (!iot::wifiOK()) return;
    if (!mqtt_.connected()) {
      if (millis() - lastTry_ < 10000) return;
      lastTry_ = millis();
      user_ = iot::setting("aiouser", AIO_USERNAME);
      String key = iot::setting("aiokey", AIO_KEY);
      if (user_.isEmpty() || key.isEmpty()) { Serial.println("[AIO] aiouser/aiokey not set"); return; }
      String id = "esp32-" + String((uint32_t)ESP.getEfuseMac(), HEX);
      Serial.println("[AIO] connecting...");
      if (mqtt_.connect(id.c_str(), user_.c_str(), key.c_str())) {
        Serial.println("[AIO] connected");
        for (int i = 0; i < nSub_; i++) {
          mqtt_.subscribe(topic(subs_[i]).c_str());
          if (requestLast_) mqtt_.publish((topic(subs_[i]) + "/get").c_str(), "");
        }
        // subscribe to throttle/error topics so we learn about rate limits
        mqtt_.subscribe((user_ + "/throttle").c_str());
      } else {
        Serial.printf("[AIO] failed, state %d\n", mqtt_.state());
      }
    }
    mqtt_.loop();
  }

  bool connected() { return mqtt_.connected(); }

  // Returns false if not connected or if the per-minute budget is used up.
  bool publish(const char* feed, const String& value) {
    if (!mqtt_.connected()) return false;
    uint32_t now = millis();
    if (now - windowStart_ > 60000) { windowStart_ = now; sent_ = 0; }
    if (sent_ >= 28) { Serial.println("[AIO] rate budget used, skipping"); return false; }
    sent_++;
    return mqtt_.publish(topic(feed).c_str(), value.c_str());
  }
  bool publish(const char* feed, float v, int dp = 2) { return publish(feed, String(v, dp)); }

 private:
  String topic(const char* feed) { return user_ + "/feeds/" + feed; }
  static AdafruitIO*& instance() { static AdafruitIO* p = nullptr; return p; }

  WiFiClientSecure tls_;
  PubSubClient mqtt_;
  FeedCallback cb_ = nullptr;
  const char* const* subs_ = nullptr;
  int nSub_ = 0;
  bool requestLast_ = true;
  String user_;
  uint32_t lastTry_ = 0, windowStart_ = 0;
  int sent_ = 0;
};

}  // namespace cloud
```